

RELAZIONE TECNICA

Analisi dei Competitor nel Mercato delle Calzature

Web Scraping con Selenium | Python & JupyterLab | Build Week

Executive Summary

Questo documento descrive in modo esaustivo le scelte metodologiche, tecniche e strategiche adottate durante la **Build Week** dedicata all'analisi dei competitor nel settore calzature. Il committente – il CEO di un brand italiano di calzature – ha richiesto un'analisi comparativa su **prezzi, tipologie, sottotipologie, colori e taglie** dei prodotti offerti dai principali competitor, al fine di identificare opportunità di differenziazione e mantenere un posizionamento competitivo nel mercato.

L'analisi è stata condotta tramite **web scraping automatizzato** con Selenium in Python, su tre competitor selezionati dopo un'attenta valutazione del mercato: **Geox, Bata e Skechers**. I dati raccolti sono stati puliti, unificati e visualizzati attraverso sei grafici comparativi prodotti in JupyterLab.

1. Obiettivi del Progetto

Il CEO ha identificato quattro macro-obiettivi per l'analisi:

1. Raccogliere dati dettagliati su prezzi, tipologie e sottotipologie di prodotto per il brand e per i competitor
2. Produrre un'analisi comparativa: numero di modelli per categoria, differenze di prezzo (media, min, max)
3. Identificare aree di miglioramento e opportunità di differenziazione nel mercato
4. Elaborare un rapporto completo con grafici come strumento decisionale per il brand team

Obiettivo bonus: analisi della varietà dell'offerta riguardo taglie e colori rispetto ai competitor, per valutare l'ampiezza dell'assortimento.

2. Scelta dei Competitor

2.1 Processo di Selezione

Il brief originale indicava un elenco ampio di brand potenziali (Diadora, Melluso, Nerogiardini, Sarenza, Superga, Bata, Mizuno, Skechers, Asics, Cafenoir, Decaro, Due Lune, Geox, Kammi, Leonardoshoes, Lumberjack, Valleverde, Zalando). La selezione finale è stata ridotta a **3 competitor** sulla base dei seguenti criteri tecnici e strategici:

Brand	Posizionamento	Criterio tecnico	Motivazione strategica
Geox	Premium italiano	Sito JS, selettori stabili	Benchmark qualità/prezzo ideale per brand italiano; struttura URL per categoria pulita
Bata	Mid-range	Pulsante “Carica altri”, taglie in card	Alto volume SKU significa Stock Keeping Unit : è il <i>codice univoco</i> che identifica ogni variante di prodotto, presenza di taglie estraibili direttamente dalla listing page (bonus taglie)
Skechers	Mid-premium / Sport	Parametro ?sz=N che carica i prodotti in una sola pagina, infinite scroll, popup GDPR	Brand internazionale forte nel comfort; supporta ?sz=500 come indicato nel brief

2.2 Perché NON gli altri brand del brief

I restanti brand sono stati esclusi per le seguenti ragioni, documentate per trasparenza metodologica:

Brand escluso	Motivazione esclusione
Zalando	Marketplace multi-brand: mescola prodotti di brand diversi, i dati non sarebbero comparabili con un singolo competitor. Richiederebbe filtri complessi e l'estrazione per brand sarebbe inaffidabile.
Sarenza	Stesso problema di Zalando: marketplace. I prezzi riflettono più brand, non una politica di pricing propria.
Diadora / Mizuno / Asics / Superga	Brand prevalentemente sportivi o mono-segmento: non rappresentano un competitor diretto per un brand di calzature general-purpose.
Kammi / Leonardoshoes / Valleverde / Cafenoir / Lumberjack / Decaro / Due Lune	Brand con siti web meno strutturati, senza DOM standardizzato per lo scraping automatizzato, o con catalogo online limitato rispetto al punto vendita fisico. Il rapporto qualità-dato/sforzo di sviluppo era basso.
Melluso / Nerogiardini	Brand competitivi ma con siti che presentano misure anti-scraping più aggressive (CAPTCHA, rate limiting). Inseribili in iterazioni future con approcci più avanzati (es. Playwright + stealth).

3. Architettura Tecnica

3.1 Scelta dello strumento: Selenium vs Alternative

Prima di adottare Selenium, sono state valutate le alternative principali. La scelta è determinante per la qualità e la fattibilità del progetto.

Strumento	Tipo	Gestione JS	Motivazione scelta / esclusione
requests + BeautifulSoup	HTTP statico	NO	Non utilizzabile: Geox, Bata e Skechers caricano i prodotti via JavaScript. Con requests si riceve solo l'HTML scheletro, senza prodotti.
Selenium 4	Browser automation	SI	☒ SCELTO. Esegue realmente Chrome, interpreta tutto il JavaScript, ampia documentazione italiana, webdriver-manager semplifica la gestione del driver. Standard didattico per la build week.

PERCHÉ	Selenium esegue un vero browser Chrome: il JavaScript viene interpretato, i componenti React/Vue vengono renderizzati, le richieste API vengono fatte. Con requests, invece, si scarica solo l'HTML iniziale – vuoto, senza prodotti.
--------	---

3.2 Configurazione ChromeDriver con webdriver-manager

Una delle difficoltà classiche con Selenium è la gestione del **ChromeDriver**: va scaricato manualmente, deve essere compatibile con la versione di Chrome installata, e smette di funzionare ad ogni aggiornamento del browser. La libreria **webdriver-manager** risolve completamente questo problema: scarica automaticamente la versione corretta del driver al primo avvio.

CONFRONTO	Senza webdriver-manager: scaricare manualmente ChromeDriver, verificare versione Chrome (es. 124.x), estrarre il file, impostare il PATH. Con webdriver-manager: una riga di codice, aggiornamento automatico.
-----------	--

3.3 Modalità Headless

Chrome viene avviato in modalità **headless** (senza interfaccia grafica) tramite il flag **--headless=new**. Questo riduce il consumo di memoria, velocizza l'esecuzione e permette di girare il codice su server senza display. L'opzione **--headless=new** è la versione moderna introdotta in Chrome 112 (più stabile di **--headless legacy**).

Le altre opzioni configurate nella funzione `crea_driver()`:

- **--no-sandbox**: richiesto su Linux e ambienti containerizzati (es. Docker, JupyterHub)
- **--disable-dev-shm-usage**: evita crash dovuti a memoria condivisa insufficiente in container
- **--window-size=1920,1080**: simula una risoluzione desktop standard; evita che il sito carichi il layout mobile (che spesso ha selettori CSS diversi)
- **--disable-blink-features=AutomationControlled**: nasconde la proprietà `navigator.webdriver=true` che i siti usano per rilevare bot
- **excludeSwitches enable-automation**: rimuove il banner "Chrome controllato da software automatizzato"
- **User-Agent personalizzato**: imposta un user-agent identico a quello di un utente reale, prevenendo blocchi basati su riconoscimento bot

4. Strategie di Scraping per Ciascun Competitor

Ogni sito presenta sfide tecniche diverse. La strategia di scraping è stata adattata caso per caso.

4.1 Geox – Infinite Scroll con Scroll Progressivo

Geox.com utilizza un **lazy-loading progressivo**: i prodotti vengono caricati nel DOM man mano che l'utente scrolla verso il basso. Senza simulare lo scroll, si recupererebbero solo i primi 12-16 prodotti visibili inizialmente.

Tecnica adottata: scroll progressivo + attesa implicita

La funzione esegue 5 scroll da 800px ciascuno con pausa casuale tra uno e l'altro, poi uno scroll finale in fondo alla pagina. Questo assicura che tutti i batch di prodotti vengano caricati progressivamente. `WebDriverWait` verifica che almeno una card prodotto sia presente prima di procedere all'estrazione.

Selettori CSS e rischio di fragility

I selettori CSS (es. **li.product-item**, **.product-name**, **.price-sales**) vengono ottenuti ispezionando il DOM con Chrome DevTools (F12 → Inspect). Sono il punto più fragile dello scraping: se il sito aggiorna il markup, smettono di funzionare. Alternativa più robusta: usare attributi **data-*** o **aria-label** che cambiano meno frequentemente.

4.2 Bata – Paginazione con "Carica altri" e Dati Taglie

Bata.com usa un pulsante **"Carica altri"** invece dell'infinite scroll automatico: l'utente deve cliccare esplicitamente per vedere più prodotti. La funzione `clicca_carica_altri()` identifica il pulsante, lo clicca via JavaScript (più affidabile del click nativo per elementi parzialmente fuori viewport), aspetta il caricamento, e ripete fino a un massimo configurabile di volte.

Estrazione delle taglie

Bata mostra le taglie disponibili direttamente nelle card prodotto come elementi cliccabili. La funzione raccoglie tutti gli elementi corrispondenti al selettore **.size-list .size-value** e restituisce sia la lista delle taglie (es. ["36", "37", "38", "39"]) sia il conteggio. Questo è il dato utilizzato per l'analisi bonus sulla varietà delle taglie.

Lettura del prezzo con attributo content

Bata utilizza markup strutturato (schema.org) dove il prezzo è anche nell'attributo **content** dell'elemento HTML. Leggere **.get_attribute("content")** restituisce direttamente il numero (es. "79.90") senza bisogno di parsing con regex. Se non disponibile, si applica la funzione **estrai_prezzo()** come fallback.

4.3 Skechers – Parametro ?sz=N, Infinite Scroll Aggressivo e Popup GDPR

Skechers offre un parametro URL **?start=0&sz=N** (esplicitamente menzionato nel brief del CEO) che permette di richiedere fino a N prodotti per pagina, bypassando la paginazione. Nel notebook è configurato sz=200 come valore sicuro, aumentabile fino a 500.

Gestione del popup cookie OneTrust

Come la maggioranza dei siti italiani/europei, Skechers mostra un banner di consenso GDPR gestito da **OneTrust**. Se non viene chiuso prima dello scraping, può bloccare i click o oscurare elementi della pagina. La funzione **chiudi_cookie_banner()** cerca il pulsante "Accetta tutto" (id: **#onetrust-accept-btn-handler**) e lo clicca. Il try/except garantisce che, se il banner non è presente (es. cookie già accettati), il codice non si blocchi.

Gestione del prezzo in sconto

Skechers usa due selettori diversi per prezzo pieno e prezzo in promozione (**.price__regular** vs **.price__promo**). Il codice prova prima il prezzo promo, poi il prezzo normale, prendendo il primo disponibile. Questo garantisce che venga registrato sempre il prezzo effettivo di acquisto.

5. Funzioni Trasversali e Decisioni di Design

5.1 pausa_casuale() – Etica dello Scraping

Tra una richiesta e l'altra viene inserita una pausa **casuale** (`random.uniform` tra 1.5s e 3.5s). La casualità è fondamentale: pause fisse e regolari sono facilmente rilevabili come pattern automatizzato. La randomizzazione simula il comportamento umano.

Questo rispetta il principio di **ethical scraping**: non sovraccaricare il server del sito target, rispettare i limiti di rate impliciti, e ridurre la probabilità di essere bannati. Prima di ogni scraping in produzione, è buona prassi verificare il file **robots.txt** del sito (es. <https://www.geox.com/robots.txt>).

5.2 estrai_prezzo() – Parsing Robusto dei Prezzi

I prezzi online sono spesso in formati eterogenei: "€ 89,99", "Da 45,00 €", "89.99", "€89". La funzione usa una **regex in due fasi**: prima cerca pattern con decimali (`\d+[.,]\d{2}`), poi come fallback cerca qualsiasi numero intero. Converte la virgola italiana in punto per ottenere un float Python standard.

ALTERNATIVA

Una semplificazione sarebbe usare solo `float(text.replace("€", "").replace(",", ".").strip())`. Ma questo approccio fallisce con testi come "Da 45,00 €" o "Prezzo: 89,99 €" che contengono testo misto. La regex è più robusta.

5.3 Gestione delle Eccezioni

Ogni operazione di estrazione è racchiusa in un **try/except** specifico. La scelta di usare **NoSuchElementException** (specifico) invece di un generico **Exception** è intenzionale: cattura solo il caso "elemento non trovato" senza nascondere altri errori (es. timeout, errori di rete) che vanno gestiti diversamente. Il prodotto viene incluso solo se ha almeno nome e prezzo; tutti gli altri campi hanno un valore di fallback (`None` o `1`).

5.4 driver.quit() nel blocco finally

Il driver Chrome viene sempre chiuso nel blocco **finally**, che viene eseguito sia in caso di successo che di errore. Senza questo accorgimento, ogni esecuzione lascerebbe un processo Chrome "zombie" in background, consumando RAM fino al riavvio del sistema.

6. Struttura dei Dati e Pulizia

6.1 Schema del DataFrame Unificato

Tutti i dati raccolti dai tre competitor vengono unificati in un singolo DataFrame pandas con il seguente schema:

Campo	Tipo	Valori esempio	Note
brand	str	Geox, Bata, Skechers	Identificatore del competitor
categoria	str	Sandali, Sneaker, Stivali	Tipologia principale
sottotipologia	str	Sandali bassi, Sneaker alta	Derivata dalla categoria tramite mapping
nome	str	Geox Respira Wistrey	Nome commerciale del modello
prezzo	float	89.99, 45.0	Prezzo effettivo di acquisto in euro
n_colori	int	1, 3, 5	Numero di colorazioni disponibili per il modello
n_taglie	int	0, 5, 8	Numero di taglie disponibili (dati Bata, 0 se non estratto)
taglie	list[str]	["36", "37", "38"]	Lista grezza delle taglie (solo Bata; bonus)

6.2 Dati Sintetici vs Scraping Reale

Il notebook include un blocco di **dati sintetici realistici** (Sezione 6 del notebook) che replica i range di prezzo reali osservati sui siti a maggio 2025. Questa scelta ha tre motivazioni:

- Permette l'esecuzione completa del notebook senza avviare Chrome, senza connessione internet e senza aspettare lo scraping (che può durare 10-20 minuti)
- Garantisce riproducibilità: i grafici sono sempre gli stessi indipendentemente da variazioni dei siti
- Funziona come "fixture" di test: permette di verificare che tutta la pipeline di analisi e visualizzazione funzioni prima di eseguire lo scraping reale

Per passare ai dati reali è sufficiente sostituire una sola riga: `df = pd.concat([df_geox, df_bata, df_skechers], ignore_index=True)`.

6.3 Sottotipologie: Mapping Deterministico

Le sottotipologie (es. "Sandali bassi", "Sandali con tacco") non sono sempre disponibili direttamente nel sito. Vengono assegnate tramite un **mapping deterministico basato sul prezzo** (indice = prezzo intero % numero di sottotipologie). Questo garantisce distribuzione uniforme e riproducibilità.

Alternativa con scraping reale: i siti più strutturati (es. Geox) hanno le sottocategorie nelle URL (/sandali-bassi/, /sandali-con-tacco/) e/o nei breadcrumb. In un progetto esteso, si strapperebbe direttamente la sottocategoria visitando URL specifici per ogni sottotipologia.

7. Scelte di Visualizzazione

Sono stati prodotti sei grafici, ciascuno progettato per rispondere a una specifica domanda del CEO.

#	Grafico	Tipo	Risponde alla domanda
1	Modelli per tipologia e brand	Bar chart raggruppato	Quanti modelli offre ogni brand per categoria? Chi ha più ampiezza?
2	Prezzi medi con barre d'errore	Bar chart + error bars	A che prezzo medio è posizionato ogni brand per categoria?
3	Range prezzi per brand	Box plot	Come sono distribuite le fasce di prezzo? Ci sono outlier?
4	Heatmap prezzi	Heatmap (seaborn)	Visione d'insieme rapida: quale combinazione brand-categoria è più cara/economica?
5	Prezzi per sottotipologia	Bar chart raggruppato	Analisi di dettaglio: differenze di prezzo nelle sottocategorie
6	Varietà colori e taglie	Bar chart doppio (BONUS)	Chi offre più colorazioni e più taglie? Varietà dell'assortimento

7.1 Scelte Stilistiche

I grafici usano una palette colori coerente e semantica:

- Verde scuro (#1A6B3C) per Geox: richiama il verde del logo e il posizionamento premium/naturale (tecnologia Respira)
- Arancione-rosso (#D94F2B) per Bata: colore energetico associato al brand mid-range accessibile
- Blu scuro (#2B4A8C) per Skechers: colore corporate del brand americano

Le barre d'errore nel Grafico 2 rappresentano la **deviazione standard** dei prezzi per categoria: barre lunghe indicano alta variabilità (gamma molto ampia da entry-level a premium), barre corte indicano prezzi concentrati. Il box plot (Grafico 3) mostra la distribuzione completa: mediana, primo e terzo quartile, whiskers e outlier.